



PONTIFICIA UNIVERSIDAD CATOLICA DE CHILE
ESCUELA DE INGENIERIA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACION

Criptografía y Seguridad Computacional - IIC3253

Tarea 4

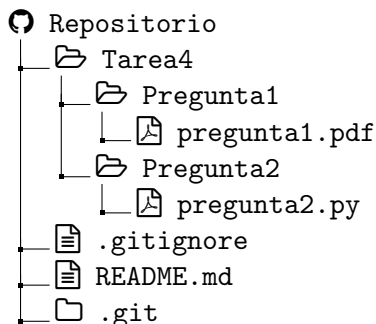
Plazo de entrega: lunes 29 de junio

Instrucciones

Cualquier duda sobre la tarea se deberá hacer en los *issues* del repositorio del curso. Los issues son el canal de comunicación oficial para todas las tareas.

Configuración inicial. Para esta tarea utilizaremos *github classroom*. Para acceder a su repositorio privado debe ingresar a un link que le enviaremos antes de la entrega de la tarea, seleccionar su nombre y aceptar la invitación. El repositorio se creará automáticamente una vez que haga esto y lo encontrará junto a los repositorios del curso. Para la corrección se utilizará Python 3.13.7.

Entrega. Al entregar esta tarea, su repositorio se deberá ver exactamente de la siguiente forma:



Para cada problema cuya solución se deba entregar como un documento, deberá entregar un archivo `.pdf` que, o bien fue construido utilizando $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$, o bien es el resultado de digitalizar un documento escrito a mano. En caso de optar por esta última opción, queda bajo su responsabilidad la legibilidad del documento. Respuestas que no puedan interpretar de forma razonable los ayudantes y profesores, ya sea por la caligrafía o la calidad de la digitalización, serán evaluadas con la nota mínima.

Firmas de anillo enlazadas

En esta tarea usted deberá implementar un protocolo **enlazable** de firmas de anillo, vale decir, un protocolo de firma de anillo donde se puede detectar si dos firmas fueron hechas por la misma persona.¹ Este protocolo estará basado en las firmas de Schnorr y el protocolo de firmas de anillo vistos en clases.

Desde ahora en adelante suponga que los siguientes objetos son públicos: un grupo $(G, *)$ con elemento neutro e , un elemento $g \in G$, un número primo q tal que $|\langle g \rangle| = q$, una función de hash h que dado un string produce un número natural, y una función de hash h_G que dado un string produce un elemento del grupo G . Por ejemplo, la función h puede ser SHA256, interpretando el output de esta función como un número natural entre 0 y $2^{256} - 1$. De la misma forma, si consideramos el grupo $(Z_n, +)$ para $n \geq 2^{256}$, entonces h_G también podría ser la función SHA256 puesto que los elementos del grupo también son números.

Recuerde que una firma de Schnorr se define de la siguiente forma. La clave secreta de un usuario A es un número $x_A \in \{1, \dots, q-1\}$ y su clave pública es el elemento del grupo $y_A = g^{x_A}$. Si el usuario A quiere generar una firma de Schnorr para un mensaje m , entonces debe realizar los siguientes pasos:

1. Genera al azar $r \in \{1, \dots, q-1\}$ y calcula $c = h(g^r \| m)$ considerando g^r como un string.
2. Calcula $s = r + c \cdot x_A$.
3. Define (c, s) como la firma de Schnorr de m .

Un usuario B puede verificar si (c, s) es una firma del mensaje m hecha por el usuario A chequeando que la siguiente condición se cumpla:

$$c = h((g^s * y_A^{q-c}) \| m).$$

Utilizando este protocolo, mostraremos cómo se define una firma de anillo enlazable para un conjunto de cuatro participantes. Posteriormente, usted deberá generalizar la construcción al caso de un número arbitrario de participantes.

Suponga que los participantes generarán firmas de anillo asociadas a un evento específico, como una encuesta o una elección presidencial. Cada evento está identificado mediante un tag t , representado por un string. Por ejemplo, el tag correspondiente a las elecciones municipales de Chile de 2028 podría ser "Elección municipal Chile 2028". Observe que eventos distintos deben utilizar etiquetas distintas, ya que estas actúan como identificadores únicos.

Además, suponga que se ha establecido un orden entre los participantes y que todas las firmas de anillo se generarán respetando dicho orden. Sean 1, 2, 3 y 4 los participantes que generarán firmas de anillo. La clave secreta del usuario i es x_i , mientras que su clave pública es $y_i = g^{x_i}$. Finalmente, suponiendo que $r = \frac{|G|}{q}$ y considerando cada clave pública y_i como un string, defina:

$$T = h_G(y_1 \| y_2 \| y_3 \| y_4 \| t)^r.$$

Dada la definición de h_G , se tiene que $T \in G$. Si $T = e$ (el elemento neutro del grupo), entonces se debe utilizar un tag distinto para el evento hasta lograr que $T \neq e$. Observe que esto asegura que $|\langle T \rangle| = q$.

¹En este protocolo no se puede identificar a la persona que hizo dos firmas de anillo. Si esto fuera posible entonces el protocolo sería rastreable.

Una vez definidos el tag t , las claves secretas y públicas de los participantes, y el elemento T del grupo, las firmas de anillo para este evento se generan de la siguiente forma. Si el usuario 2 quiere generar una firma de anillo de un mensaje m , define $f = T^{x_2}$ y realiza los siguientes pasos:

1. Genera al azar $r_2 \in \{1, \dots, q-1\}$ y calcula $c_3 = h(g^{r_2} \| m \| T^{r_2})$.
2. Genera al azar $s_3 \in \{1, \dots, q-1\}$.
3. Calcula $c_4 = h((g^{s_3} * y_3^{q-c_3}) \| m \| (T^{s_3} * f^{q-c_3}))$.
4. Genera al azar $s_4 \in \{1, \dots, q-1\}$.
5. Calcula $c_1 = h((g^{s_4} * y_4^{q-c_4}) \| m \| (T^{s_4} * f^{q-c_4}))$.
6. Genera al azar $s_1 \in \{1, \dots, q-1\}$.
7. Calcula $c_2 = h((g^{s_1} * y_1^{q-c_1}) \| m \| (T^{s_1} * f^{q-c_1}))$.
8. Calcula $s_2 = r_2 + c_2 \cdot x_2$. Note que este paso lo puede realizar el usuario 2 ya que conoce la clave secreta x_2 .
9. Define $(s_1, s_2, s_3, s_4, c_1, f)$ como la firma de anillo de m . En particular, siempre se incluye c_1 en la firma, independientemente de quién la haya generado.

Note que cuando un participante genera una firma de anillo debe respetar el orden preestablecido. Por ejemplo, si el usuario 3 quiere firmar, entonces debe realizar el protocolo en el orden $3 \rightarrow 4 \rightarrow 1 \rightarrow 2 \rightarrow 3$. Además, note que como fue mencionado en clases, basta que la firma incluya los valores s_1, s_2, s_3, s_4 y un valor c_i para verificar que es correcta. En este caso, este valor siempre será c_1 , de manera tal de no entregar información sobre quién generó la firma. Finalmente, observe que se incluye en la firma un valor f adicional que será utilizado para la enlazabilidad.

Si un usuario B quiere verificar que $(s_1, s_2, s_3, s_4, c_1, f)$ es una firma de anillo válida para el tag t y el grupo formado por los usuarios 1, 2, 3 y 4, entonces debe hacer lo siguiente:

1. Calcula $T = h_G(y_1 \| y_2 \| y_3 \| y_4 \| t)^r$, donde $r = \frac{|G|}{q}$
2. Calcula $c_2 = h((g^{s_1} * y_1^{q-c_1}) \| m \| (T^{s_1} * f^{q-c_1}))$
3. Calcula $c_3 = h((g^{s_2} * y_2^{q-c_2}) \| m \| (T^{s_2} * f^{q-c_2}))$
4. Calcula $c_4 = h((g^{s_3} * y_3^{q-c_3}) \| m \| (T^{s_3} * f^{q-c_3}))$
5. Verifica si $h((g^{s_4} * y_4^{q-c_4}) \| m \| (T^{s_4} * f^{q-c_4}))$ es igual al valor c_1 de la firma.

Si esta última condición es cierta, entonces B sabe que la firma es válida, pero no sabe cuál de los usuarios firmó el mensaje m .

Además, B puede verificar si dos firmas fueron hechas por el mismo participante de la siguiente forma. Si un usuario i firma dos mensajes m_1 y m_2 , entonces la firma de m_1 es de la forma $(s_1, s_2, s_3, s_4, c_1, f)$ y la firma de m_2 es de la forma $(s'_1, s'_2, s'_3, s'_4, c'_1, f)$, donde en ambos casos $f = T^{x_i}$. De hecho, todas las firmas de anillo del usuario i para este evento y este grupo van a tener como último componente $f = T^{x_i}$. De esta forma, B puede verificar si dos firmas fueron

hechas por el mismo participante simplemente chequeando que las últimas componentes en estas firmas son iguales.

Finalmente, observe que el protocolo anterior sólo nos permite verificar si dos firmas de anillo fueron generadas por el mismo usuario en el mismo evento. En particular, un participante puede generar firmas de anillo para eventos distintos sin que ellas sean identificadas como hechas por la misma persona. En efecto, si consideramos dos eventos distintos con tags $t_1 \neq t_2$, entonces esperamos que $T_1 \neq T_2$ puesto que estamos usando una función de hash h_G que es resistente a colisiones. De esta forma, una firma de anillo de un participante i en el primer evento es de la forma $(s_1, s_2, s_3, s_4, c_1, f_1)$ con $f_1 = T_1^{x_i}$, mientras que una firma de anillo en el segundo evento es de la forma $(s'_1, s'_2, s'_3, s'_4, c'_1, f_2)$ con $f_2 = T_2^{x_i}$. Como $f_1 \neq f_2$, puesto que $T_1 \neq T_2$, estas firmas no pueden ser reconocidas como hechas por el mismo participante.

Preguntas

1. Responda las siguientes preguntas.

- [0.5 puntos] Para el caso de cuatro participantes, demuestre que si el participante 3 construye una firma $(s_1, s_2, s_3, s_4, c_1, f)$ de un mensaje m utilizando el protocolo, entonces el proceso de verificación concluye que la firma es válida.
- [0.5 puntos] Un participante i podría tratar de ocultar que generó dos firmas utilizando $f = T^{x_i}$ en una de las firmas, y un elemento arbitrario $f' \in G$ con $f' \neq f$ en la otra firma. Considerando nuevamente el caso con cuatro participantes, explique por qué el participante 1 no podría generar una firma válida si utiliza un elemento $f' \in G$ tal que $f' \neq T^{x_1}$.
- [0.5 puntos] Demuestre que $|\langle T \rangle| = q$.
- [0.5 puntos] Suponga que modificamos el protocolo de manera que el elemento T usado en el protocolo es generado por una tercera parte; por ejemplo, este elemento T podría ser generado por el Servel en caso de una elección o por los profesores del curso en el caso de esta tarea. Muestre que, en este caso, el protocolo puede ser quebrado. En particular, muestre que esta tercera parte puede generar eficientemente un elemento $T \in G$ tal que $|\langle T \rangle| = q$, $T \neq g$ y, si T se utiliza en la generación de firmas de anillo, entonces puede determinar eficientemente quién generó cada firma.

2. [2 puntos] En esta pregunta usted deberá programar una clase que represente a un participante de un conjunto de personas que generan firmas de anillos trazables. También una clase que revise dichas firmas y que verifique si el participante que emitió la firma ya había emitido otra firma anteriormente. Utilizaremos siempre SHA256 y grupos multiplicativos de enteros módulo primos $(\mathbb{Z}_p^*)$. En particular:

- Para transformar un número n a un string, simplemente utilizaremos `str(n)`.
- Para transformar el hash de un string s a un elemento del grupo $\mathbb{Z}_p^*$ utilizaremos `int.from_bytes(sha256(s.encode())) % p`.

La clase que representa a un participante debe tener la siguiente interfaz:

```

class TraceableRingSignatureParticipant:
    def __init__(self, g: int, q: int, p: int) -> None:
        # Verify that p is prime, raise an AssertionError otherwise.
        # Verify that q is prime in  $\mathbb{Z}_p^*$ , raise an AssertionError otherwise.
        # Verify that the order of the subgroup generated by g is q, raise an
            AssertionError otherwise.

        # Randomly generate the participant's secret key x in  $\{1..q\}$ 

    def get_public_key(self) -> int:
        # Returns the participant's public key ( $g^x \bmod p$ )

    def generate_traceable_ring_signature(self, public_keys: list[int], tag: str
        , message: str) -> Tuple[list[int],
            int, int]:
        # Compute  $T := (\text{SHA256}(p_1 || \dots || p_n || \text{tag})^r \bmod p)$ , where  $[p_1, \dots, p_n]$  is the list public_keys of
            public keys.

        # Verify that the order of the subgroup generated by T is q, raise an
            AssertionError otherwise.

        # Generate a traceable ring signature for the given message, tag and list
            of public keys.

        # The output looks like  $([s_1, \dots, s_n], c_1, f)$ .

```

La clase que representa a un verificador debe tener la siguiente interfaz:

```

class Verifier:
    def __init__(self, g: int, q: int, p: int) -> None:
        # Verify that p is prime, raise an AssertionError otherwise.
        # Verify that q is prime in  $\mathbb{Z}_p^*$ , raise an AssertionError otherwise.
        # Verify that the order of the subgroup generated by g is q, raise an
            AssertionError otherwise.

    def verify_signature(self, public_keys: list[int], message: str, tag: str,
        signature: Tuple[list[int], int, int
            ]) -> bool, bool:
        # Verify if the signature is a valid traceable ring signature for the
            given public keys, message and tag.

        # The first component of the output is True if the signature is valid.
        # The second component is True if this is the first signature of this
            participant seen by the verifier.

        # Note the verifier needs to store some state to compute the second
            component.

```

3. En esta pregunta usted deberá utilizar su código de la pregunta 2 para participar en una encuesta anónima (pero en serio) sobre este curso. Utilizaremos el tag IIC3253-2026 y el grupo $\mathbb{Z}_p^*$ con los siguientes parámetros:

g = 80413673270461893026939846650267063748446082898743744257287976695094358814591
 40662650215832833471328470334064628508692231999401840332046192569287351991689
 96327965689256248477327858420804098763156962852046406953236127404737444434499
 66518329793783188499437416621103959957784292708192224316109273560059138369324
 62099770076239554042855287138026806960470277326229482818003962004453764400995
 79097404266367569212075872614586906123644389350913614794241444555184816239146
 85414443557077856978257418568491612338873070174283718236081256998929049608412
 21593344499088996021883972185241854777608212592397013510086894908468466292313
 q = 63762351364972653564641699529205510489263266834182771617563631363277932854227
 p = 17125458317614137930196041979257577826408832324037508573393292981642667139747
 62177880243877523872859296834461358937993234847561350347693216316697381321869
 83438164632891441853629126025225404949830905314972329658295365245072698488256
 58311420299335922295709743267508322525966773950394919257576842038771632742044
 14247105350985012360588381585716266691777519349615737265619555830572700989127
 60065140004093658772181713883199238963093777917625906143118496429613802248519
 40460421710449368927252974870395873936387909672274883295377481008150475878590
 270591798350563488168080923804611822387520198054002990623911454389104774092183

- (a) [0.5 puntos] El día miércoles 17 de junio se publicará en el issue de anuncios del curso una URL a la cual usted debe enviar un request POST cuyo *body* siga el siguiente esquema:

`{public_key: number, hmac: string}`

`public_key` es su llave pública para participar en la encuesta. `hmac` es la representación hexadecimal de `HMAC-SHA256(symmetric_key, public_key)`, donde `symmetric_key` es la llave simétrica que hemos utilizado a lo largo del curso para encriptar sus notas.

Este endpoint responderá con los siguientes códigos:

- 201 Su request se procesa correctamente.
- 422 La llave pública o el `hmac` no están en el formato correcto.
- 401 El input está en el formato correcto pero el `hmac` no es válido.
- 409 Todo es correcto pero ya se había enviado una llave pública cuyo `hmac` se había generado con esa llave simétrica.
- 403 Todo es correcto pero esa llave pública ya fue inscrita anteriormente con otro `hmac`.

Usted debe enviar un request al cual se responda con código 201 antes del viernes 19 de junio a las 11:59pm.

Importante: No habrá segundas oportunidades, cuide su llave secreta.

- (b) [0.5 puntos] El día sábado 20 de junio se publicará en el issue de anuncios del curso una URL para obtener, en orden, todas las llaves públicas que se enviaron de acuerdo a la pregunta anterior. El lunes 22 se publicará URL para *votar* en la encuesta del curso. Usted deberá enviar un request POST a esta URL con un *body* que siga el siguiente esquema:

```
{
  voto: {nps: number, chachara: string, comentario: string},
  firma: [s: [number], c: number, f: number]},
}
```

- **nps** debe ser un número entero entre 1 y 10 que responda a la pregunta: “De 1 a 10, qué puntaje le pondría a este curso”, donde 1 es el menor puntaje y 10 el mayor.
- **chachara** debe ser un string en el conjunto {marcelo,martin} que indique cuál de los dos profesores explica de forma más extensa (le mete más cháchara).
- **comentario** es un string de a lo más 500 caracteres en el que usted puede dejar los comentarios que quiera, con la única restricción de que no se debe hacer referencia a otros estudiantes o profesores de la universidad que no sean los del curso. En tal caso su voto no será considerado ni tendrá oportunidad de enviar otro voto.
- **firma** es una firma de anillo enlazable de acuerdo a lo descrito en la pregunta anterior y el tag IIC3253-2026. El mensaje a firmar es el **voto** en formato JSON interpretado como un string, formateado sin saltos de línea, sin espacios fuera del comentario y con comillas dobles, exactamente como se muestra en el siguiente ejemplo:

```
{nps:10,chachara:"marcelo",comentario:"muy buen curso"}
```

El endpoint responderá con los siguientes códigos:

201 El voto es recibido correctamente.

422 El esquema de su request es inválido.

401 El esquema de su request es válido pero la firma es inválida.

409 El esquema es válido y la firma es válida, pero dicha firma se generó con una llave secreta con la que se había firmado y enviado otro voto válido anteriormente.

Para que su voto sea considerado debe enviar un request cuya respuesta tenga código 201.

- (c) [1 punto] En esta pregunta grupal, usted obtendrá puntaje de acuerdo a cómo se comporte el curso al responder la encuesta. Suponga que el número de personas que enviaron su clave pública es M , y que el número de encuestas válidas recibidas hasta la fecha de entrega de la tarea es V . Si usted envió su clave pública dentro del plazo, entonces recibirá $\frac{V}{M}$ puntos en esta pregunta.